Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Experiencia de Aprendizaje (EA3) - Proyecto de Aula (Fase 3)

Implementación de la estructura completa de una base de datos (DDL)

- **Tarea en Equipo**
- **Peso: 10%**
- **Práctica. Caso de Estudio "Red Social Pascualina"**
- **Diccionario de Datos Físico - DBMS - PostgreSQL - pgAdmin4**

MIEMBROS DEL EQUIPO:

- Estudiante: Juan Camilo González Hoyos

Descripción de la Tarea

1. Contextualización

Hemos identificado una necesidad clara: mejorar la comunicación y la interacción entre los estudiantes. Aunque tenemos plataformas institucionales, a menudo son formales y no facilitan la conexión casual, el intercambio de conocimientos entre pares, la formación de grupos de estudio o la organización de eventos informales.

La comunidad educativa necesita una plataforma donde puedan **(SI NO ESTÁN, AGREGAR AL MODELO):**

- **Conectar con compañeros**: encontrar y seguir a otros estudiantes con intereses similares o de cursos avanzados que puedan ofrecer mentoría.
- **Publicar y comentar actualizaciones**: compartir preguntas sobre tareas, recursos útiles, noticias de la industria tecnológica o incluso memes relevantes para la vida universitaria.
- **Crear y unirse a grupos**: organizar grupos de estudio para materias específicas, equipos para hackatones o clubes de interés (videojuegos, programación competitiva, etc.).
- **Comprar o vender artículos o servicios**: todos los usuarios podrán realizar estas operaciones. Ejemplo: un estudiante o miembro de la Red (tipo de usuario) vende cuadernos de "Hello Kitty" (producto), que puede ser tipificado como "papelería" (tipo de producto); y debe quedar "traza" de las ventas del producto (decida si es una relación o uan nueva entidad). NOTA: Esto aplica para servicios y eventos.
- **Realizar actividades relacionadas con nuevas tablas agregadas por el equipo de estudiantes**: los usuarios podrán realizar estas operaciones según su perfil

El objetivo con este reto es diseñar la base de datos que soportará la Red Social que Ud. diseño con estas características. Este es un problema real que resolverán usando las herramientas y conceptos aprendidos. El objetivo de este reto es tomar el modelo lógico que ya crearon para la red social estudiantil pascualina y lo implementen como un modelo físico funcional en un Sistema de gestión de base de datos (SGBD) real, aplicando rigurosamente las reglas y estándares del lenguaje de definición de datos (DDL).

Para lograrlo, aplicarán todo lo aprendido con el libro ***"Estructura, reglas y estándares"*,** que hace parte de los recursos educativos de la unidad; así como todo el material compartido por el docente y la plataforma.

Es importante al identificación correcta de los tipos de datos del SGBD en el cual implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario físico debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos genéricos en los tipos de datos del SGBD.

2. Instrucciones de realización de la tarea

Según los requerimientos que se planteen, se puede seleccionar un SGBD en particulazr que consideren más adecuado, de acuerdo con las necesidades de una base de dato. Luego, utilizando una herramienta gráfica, como pgAdmin4 o MySQL Workbench, el estudiante se debe enfocar en la correcta implementación del modelo, prestando especial atención a la identificación de tablas, campos, los tipos y tamaños de datos, y, fundamentalmente, la definición de las claves primarias y las relaciones foráneas que dan coherencia al sistema. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final.

NOTA: En esta tarea se UTILIZARÁ OBLIGATORIAMENTE PostgreSQL a través de la herramienta gráfico pgAdmin4.

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones.
- Diccionario de Datos Físico (hoja de cálculo)
- Los Scripts con el modelo físico (DDL). Nota: Creación y Modificación de Tablas
- Un video corto donde sustenten el trabajo realizado, muestren instrucciones en "ejecución" en vivo en eol ambiente gráfico del SGBD y conclusiones individuales breves que reflejen su aprendizaje personal.

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DDL y la implementación de una base de datos en un SGBD, habilidades fundamentales para cualquier desarrollador de software.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura:
Carpeta raíz: Tarea3. En esta carpeta estarán dos carpetas con los entregables: Informe y Video.

Carpetas	Contenido requerido	Observaciones
/tarea3	Todas las subcarpetas y archivos de la tarea.	No aplica
tarea3/informe	Informe detallado	.pdf
tarea3/Resultados	Diccionario de Datos Físico (hoja de cálculo)	.xlsx
	Script de creación y Script de modificación de la base de datos (archivos planos con instrucciones DDL)	.sql
tarea3/video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y ejectudando instrucciones en el SGBD	Enlace a YouTube

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: ● Descripción del modelo físico en el que expliquen cómo el modelo lógico fue transformado. ● Descripción de cada tabla implementada con: nombre de la tabla, listado de campos (atributos) con su tipo de dato y tamaño (si aplica), justificación de la elección de tipos de datos para campos clave (por ejemplo, por qué eligieron VARCHAR en lugar de TEXT para un campo específico, o INT en lugar de BIGINT), identificación de la clave primaria (PK) y justificación de su elección. ● Descripción de cómo se implementaron las relaciones entre tablas. Para cada relación incluir: tablas involucradas (tabla padre, tabla hija y(o tabla pivote), campos que conforman la clave foránea, especificación de las acciones referenciales (ON DELETE, ON UPDATE, ON CASCADE) y su justificación. ● El informe debe referenciar el script DDL y explicar su estructura. ● Conclusiones individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.
Modelo físico con: ● Inventario completo de tablas de la base de datos. ● Tablas con nombres según lasbuenas prácticas, reglas y la nomenclatura. ● Campos correctamente identificados con tamaño y tipo asociado. ● Relaciones adecuadas al caso (claves primarias y foráneas). ● Campos con el uso correcto las reglas, nomenclatura y estándares para BD físicas.	Comprensión del modelo físico

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencia	Habilidad evaluada
Video de sustentación	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

Analicen los criterios de evaluación y verifiquen su cumplimiento..

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

1.- Revisión Modelo Conceptual (corregido/mejorado)

1.1.- Descripción de los cambios realizados al Modelo Conceptual original después de la actividad de co-evaluación con los modelos de los otros estudiantes.

Después de la co-evaluación con los modelos de los otros equipos y de la revisión de la nota de la Fase 2, el equipo comparó el modelo conceptual (EA1), el modelo lógico normalizado (EA2) y las nuevas exigencias del enunciado de la EA3. Se encontró que el modelo lógico de la Fase 2 se había alejado del conceptual: varias relaciones 1:N se convirtieron en tablas independientes que repetían los atributos de la entidad, lo que llevó el inventario a 46 tablas con información duplicada. Las correcciones realizadas fueron las siguientes:

1. **Eliminación de tablas duplicadas por relaciones 1:N.** Tablas como usuario_envia_mensaje, usuario_realiza_publicacion, usuario_vende_producto, usuario_ofrece_servicio y usuario_organiza_evento repetían los mismos campos de mensaje, publicacion, producto, servicio y evento. Una relación 1:N no genera tabla: se implementa con una clave foránea en la tabla del lado N. Se eliminaron 23 tablas redundantes y sus datos quedaron como FK (por ejemplo, publicacion.id_usuario).

2. **Doble representación de rol y tipo de usuario.** usuario ya tenía id_rol e id_tipo_usuario y además existían usuario_tiene_rol y usuario_tiene_tipo. Esto permitía que ambas fuentes se contradijeran. Como ambas relaciones son N:1, se conservan solo las FK en usuario.

3. **Clave foránea a una columna inexistente en cuenta.** usuario_tiene_cuenta y cuenta_tiene_perfil referenciaban un id_cuenta que no existe, porque la PK de cuenta es id_usuario (entidad débil 1:1). Se eliminaron ambas tablas: cuenta usa id_usuario como PK y FK al mismo tiempo.

4. **Referencia circular entre usuario y tipo_usuario.** usuario apunta a tipo_usuario y tipo_usuario apuntaba a usuario (id_usuario_registra). En el modelo físico esto impide crear y poblar las tablas en orden. tipo_usuario y rol quedan como catálogos independientes cargados por el sistema; la auditoría de quien registra se mantiene en tipo_servicio, tipo_producto y tipo_evento.

5. **Perfil de usuario y de grupo.** Se simplificó la cadena usuario-cuenta-perfil: perfil referencia directamente a usuario o a grupo, con un CHECK que obliga a que pertenezca a uno de los dos y nunca a ambos. Así se implementa la regla de exclusividad que quedó pendiente en la EA1.

6. **Membresía de grupos unificada.** usuario_solicita_grupo y usuario_administra_grupo duplicaban la relación usuario-grupo. Se integran en grupo_usuario mediante estado_membresia (pendiente, activa, rechazada, retirada) y rol_en_grupo (administrador, moderador, miembro).

7. **Relación para seguir usuarios en el diagrama.** El enunciado de la EA3 pide encontrar y seguir a otros estudiantes. La tabla usuario_sigue_usuario existía en la EA2 pero no en el diagrama de la EA1; se agrega al diagrama como relación recursiva N:M y en el modelo físico se le añade un CHECK que impide que un usuario se siga a sí mismo.

8. **Relaciones N:M omitidas en la EA2.** El modelo conceptual de la EA1 tenía las relaciones solicita (usuario-servicio), compra (usuario-producto) y participa (usuario-evento), pero la normalización de la EA2 no las convirtió en tablas. Se recuperan como solicitud_servicio, compra_producto y participacion_evento, que además son la traza de ventas que exige la EA3.

9. Traza de ventas de productos, servicios y eventos. Se decidió modelar la compra y la solicitud como entidades asociativas con clave propia (compra_producto y solicitud_servicio) y no como relaciones con PK compuesta, porque un mismo usuario puede comprar el mismo producto o contratar el mismo servicio varias veces. En eventos, participacion_evento registra valor_pagado y evento incorpora valor_entrada.

10. Calificación ligada a la contratación. calificacion pasó de (id_servicio, id_usuario) a depender de solicitud_servicio: solo puede calificar quien contrató el servicio y se permite una calificación por contratación.

11. Clave del comentario. La clave compuesta (id_publicacion, consecutivo_comentario) obligaba a calcular el consecutivo en la aplicación, con riesgo de duplicados cuando dos usuarios comentan a la vez. Se reemplaza por una PK sustituta id_comentario generada por el SGBD, conservando la FK a publicacion.

12. Publicaciones dentro de grupos. Se agrega la relación grupo contiene publicacion (FK opcional id_grupo) para que los grupos de estudio tengan su propio muro, como lo describe el caso.

13. Nombres de atributos unificados. Se corrigieron nombres inconsistentes entre fases (contenido_mensaje/cuerpo_mensaje, puntaje_estrellas/puntuacion, precio_unidad/precio_producto, titulo_evento/nombre_evento) y se aplicó snake_case singular en todo el modelo.

14. Ajustes tras la revisión con el docente. Se eliminó la columna uuid_usuario: la PK entera ya identifica al usuario y el UUID ocupaba 16 bytes sin necesidad. Además, todo atributo de solo dos valores pasó a BOOLEAN: condicion_producto ('nuevo'/'usado') se reemplazó por es_nuevo. Los estados con tres o más valores se mantienen como VARCHAR con CHECK.

15. Diagrama E-R rehecho. El diagrama anterior no coincidía con el inventario de relaciones. Se rehízo para que cada rombo corresponda a una relación del inventario, con entidades débiles en rectángulo doble, relaciones identificadoras en rombo doble y la cardinalidad (1:1 o 1:N) escrita a ambos lados de cada relación, con la misma notación del diagrama de la EA1, como lo pide el docente.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

1.2- Diagrama Entidad-Relación de Chen (corregido/mejorado)

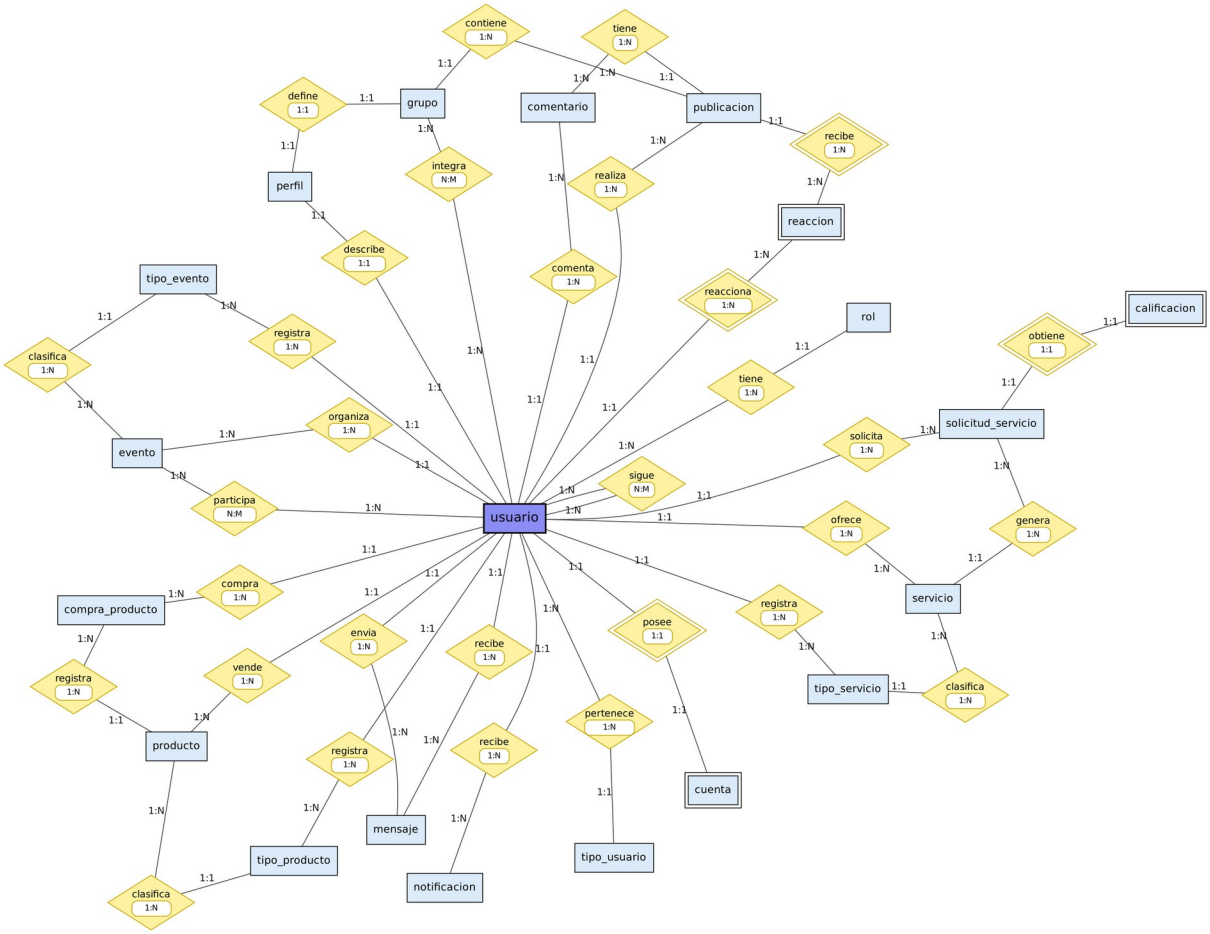


Figura 1. Diagrama Entidad-Relación (Chen) corregido, sin atributos. Rectángulo doble: entidad débil; rombo doble: relación identificadora. La entidad usuario se resalta por ser el eje del modelo.

Notación (la misma del diagrama de la EA1): cada rombo contiene el verbo de la relación y, en una etiqueta redondeada, su tipo global (1:1, 1:N o N:M). Sobre cada extremo de la línea se indica la cardinalidad con la que participa la entidad de ese lado (1:1 o 1:N). Por ejemplo, usuario 1:1 — realiza [1:N] — 1:N publicacion: cada publicación tiene un único autor y un usuario puede realizar muchas publicaciones. El modelo corregido tiene 20 entidades (17 fuertes y 3 débiles: cuenta, reaccion y calificacion) y 31 relaciones. Las relaciones N:M con atributos propios son integra (grupo_usuario), sigue (usuario_sigue_usuario) y participa (participacion_evento). Las transacciones de compra y contratación se representan como entidades (compra_producto y solicitud_servicio) porque tienen identidad propia y pueden repetirse entre el mismo par de usuario y producto o servicio.

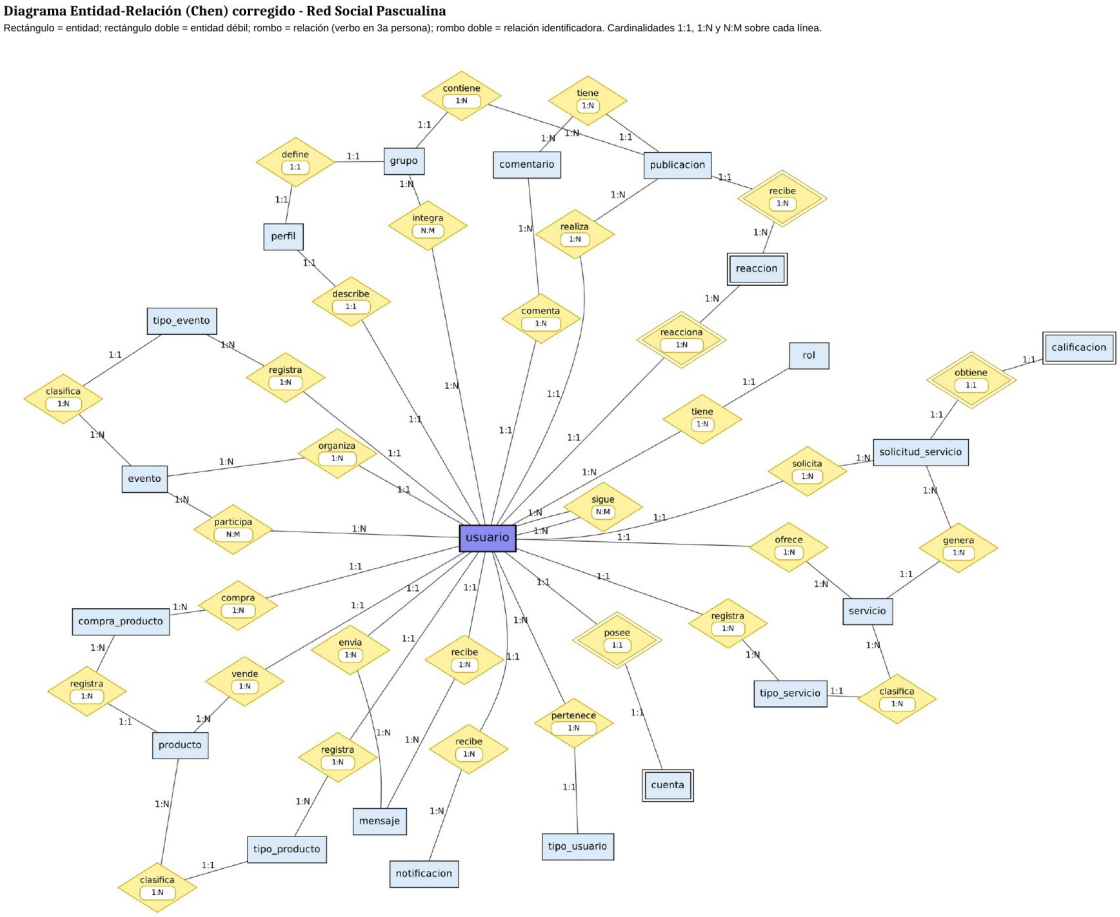


Figura 2. Pestaña "Diagrama ER (corregido)" de la hoja de cálculo de resultados.

2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

El modelo lógico corregido queda en 23 tablas en 3FN: 20 provienen de entidades y 3 de relaciones N:M con atributos (grupo_usuario, usuario_sigue_usuario y participacion_evento). Todas las relaciones 1:N y 1:1 se resolvieron con claves foráneas, por lo que ninguna tabla repite atributos de otra.

#	Tabla	Descripción	Origen
1	rol	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).	Entidad
2	tipo_usuario	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).	Entidad
3	usuario	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.	Entidad
4	cuenta	Credenciales y estado de acceso del usuario, separadas de sus datos personales.	Entidad débil (1:1)
5	grupo	Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.	Entidad
6	perfil	Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.	Entidad
7	grupo_usuario	Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.	Relación N:M
8	usuario_sigue_usuario	Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).	Relación N:M recursiva
9	publicacion	Contenido que un usuario comparte en la Red, opcionalmente dentro	Entidad

#	Tabla	Descripción	Origen
		de un grupo.	
10	comentario	Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.	Entidad
11	reaccion	Reacción breve (me gusta, me encanta…) de un usuario a una publicación; una sola por usuario y publicación.	Entidad débil
12	mensaje	Mensaje privado enviado por un usuario a otro dentro de la plataforma.	Entidad
13	notificacion	Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.	Entidad
14	tipo_servicio	Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC…).	Entidad
15	servicio	Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.	Entidad
16	solicitud_servicio	Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.	Relación N:M con atributos (entidad asociativa)
17	calificacion	Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.	Entidad débil
18	tipo_producto	Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets…).	Entidad
19	producto	Artículo que un usuario publica para la venta, con precio, existencias y condición.	Entidad
20	compra_producto	Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.	Relación N:M con atributos (entidad asociativa)
21	tipo_evento	Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto…).	Entidad
22	evento	Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.	Entidad
23	participacion_evento	Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.	Relación N:M con atributos

Tablas resultantes del Diccionario Genérico (Modelo Lógico)			
#	Tabla	Descripción Tabla	Observaciones
1	rol	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).	Entidad. Catálogo independiente. Se elimina el FK a usuario para evitar referencia circular.
2	tipo_usuario	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).	Entidad. Catálogo independiente. Se elimina id_usuario_registra (referencia circular con usuario).
3	usuario	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.	Entidad. Absorbe las relaciones N:1 'tiene rol' y 'tiene tipo' como FK. Se eliminan usuario_tiene_rol y usuario_tiene_tipo.
4	cuenta	Credenciales y estado de acceso del usuario, separadas de sus datos personales.	Entidad débil (1:1). PK = FK a usuario. Se elimina usuario_tiene_cuenta (duplicaba la relación 1:1).
5	grupo	Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.	Entidad. Se agregan es_privado y estado_grupo del modelo conceptual.
6	perfil	Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.	Entidad. Absorbe cuenta_tiene_perfil y grupo_define_perfil. CHECK de exclusividad usuario/grupo.
7	grupo_usuario	Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.	Relación N:M. Absorbe usuario_solicita_grupo y usuario_administra_grupo (estado_membresia y rol_en_grupo).
8	usuario_sigue_usuario	Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).	Relación N:M recursiva. Nueva exigencia de la EA3: conectar y seguir compañeros.
9	publicacion	Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo.	Entidad. Absorbe usuario_realiza_publicacion (era la misma tabla duplicada).
10	comentario	Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.	Entidad. Absorbe publicacion_tiene_comentario y usuario_comenta.
11	reaccion	Reacción breve (me gusta, me encanta...) de un usuario a una publicación; una sola por usuario y publicación.	Entidad débil. PK compuesta (id_publicacion, id_usuario). Absorbe publicacion_tiene_reaccion y usuario_reacciona.
12	mensaje	Mensaje privado enviado por un usuario a otro dentro de la plataforma.	Entidad. Absorbe usuario_envia_mensaje y usuario_recibe_mensaje.
13	notificacion	Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.	Entidad. Absorbe usuario_recibe_notificacion.
14	tipo_servicio	Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC...).	Entidad. Absorbe usuario_registra_tipo_servicio como FK id_usuario_registra.
15	servicio	Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.	Entidad. Absorbe usuario_ofrece_servicio y tipo_servicio_tiene_servicio como FK.
16	solicitud_servicio	Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.	Relación N:M con atributos (entidad asociativa). Implementa la relación 'solicita' y la traza de ventas de servicios exigida por la EA3.
17	calificacion	Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.	Entidad débil. Se liga a la solicitud para que solo califique quien contrató. Absorbe servicio_recibe_calificacion y usuario_califica.
18	tipo_producto	Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets...).	Entidad. Absorbe usuario_registra_tipo_producto como FK id_usuario_registra.
19	producto	Artículo que un usuario publica para la venta, con precio, existencias y condición.	Entidad. Absorbe usuario_vende_producto y tipo_producto_tiene_producto como FK.
20	compra_producto	Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.	Relación N:M con atributos (entidad asociativa). Implementa la relación 'compra' y la traza de ventas de productos exigida por la EA3.
21	tipo_evento	Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto...).	Entidad. Absorbe usuario_registra_tipo_evento como FK id_usuario_registra.
22	evento	Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.	Entidad. Absorbe usuario_organiza_evento y tipo_evento_tiene_evento como FK.
23	participacion_evento	Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.	Relación N:M con atributos. Implementa la relación 'participa' y la traza de ventas de eventos.

Figura 3. Pestaña "Inventario Modelo Lógico (corregido)" de la hoja de cálculo.

Trazabilidad del modelo lógico anterior (EA2) al corregido

Para garantizar que no se perdió información del modelo lógico anterior, la siguiente tabla muestra el destino de cada una de las 46 tablas de la EA2. Las 18 entidades y las N:M se conservan; las tablas que representaban relaciones 1:1 y 1:N se convirtieron en claves foráneas, porque en el modelo relacional una relación 1:N no genera tabla. Ningún atributo se pierde. Además, se recuperan tres relaciones N:M del modelo conceptual (EA1) que la EA2 había omitido.

#	Tabla EA2	Destino en EA3	Cómo / por qué
1	usuario	usuario	Se conserva (entidad).
2	rol	rol	Se conserva (catálogo).
3	tipo_usuario	tipo_usuario	Se conserva; se quita id_usuario_registra (ciclo con usuario).
4	cuenta	cuenta	Se conserva; PK = FK id_usuario.
5	perfil	perfil	Se conserva; recupera los atributos de la EA1 (area_estudio, atributos_perfil JSONB).
6	grupo	grupo	Se conserva.
7	mensaje	mensaje	Se conserva; recibe las FK de remitente y destinatario.
8	publicacion	publicacion	Se conserva.

#	Tabla EA2	Destino en EA3	Cómo / por qué
9	comentario	comentario	Se conserva.
10	reaccion	reaccion	Se conserva; PK compuesta (id_publicacion, id_usuario).
11	notificacion	notificacion	Se conserva.
12	tipo_servicio	tipo_servicio	Se conserva.
13	servicio	servicio	Se conserva.
14	calificacion	calificacion	Se conserva; ahora depende de solicitud_servicio.
15	tipo_producto	tipo_producto	Se conserva.
16	producto	producto	Se conserva.
17	tipo_evento	tipo_evento	Se conserva.
18	evento	evento	Se conserva.
19	grupo_usuario	grupo_usuario	Se conserva (N:M).
20	usuario_solicita_grupo	grupo_usuario	Integrada: estado_membresia = pendiente.
21	usuario_administra_grupo	grupo_usuario	Integrada: rol_en_grupo = administrador.
22	usuario_sigue_usuario	usuario_sigue_usuario	Se conserva (N:M recursiva).
23	usuario_tiene_cuenta	cuenta.id_usuario	1:1 → FK. Además referenciaba cuenta.id_cuenta, columna que no existe (error en PostgreSQL).
24	cuenta_tiene_perfil	perfil.id_usuario	1:1 → FK. También usaba id_cuenta inexistente.
25	grupo_define_perfil	perfil.id_grupo	1:1 → FK.
26	usuario_tiene_rol	usuario.id_rol	N:1 → FK; usuario ya tenía id_rol (dato duplicado).
27	usuario_tiene_tipo	usuario.id_tipo_usuario	N:1 → FK; usuario ya tenía id_tipo_usuario (dato duplicado).
28	usuario_envia_mensaje	mensaje.id_usuario_remitente	1:N → FK; repetía asunto, contenido y fecha de mensaje.
29	usuario_recibe_mensaje	mensaje.id_usuario_destinatario	1:N → FK; estado de lectura = mensaje.fue_leido.
30	usuario_realiza_publicacion	publicacion.id_usuario	1:N → FK; repetía texto, multimedia, fecha y estado.
31	publicacion_tiene_comentario	comentario.id_publicacion	1:N → FK.
32	usuario_comenta	comentario.id_usuario	1:N → FK; repetía texto, fecha y estado.
33	publicacion_tiene_reaccion	reaccion.id_publicacion	1:N → FK (parte de la PK).
34	usuario_reacciona	reaccion.id_usuario	1:N → FK (parte de la PK); repetía tipo y fecha.
35	usuario_recibe_notificacion	notificacion.id_usuario	1:N → FK; repetía título, mensaje y fecha.
36	usuario_registra_tipo_servicio	tipo_servicio.id_usuario_registra	1:N → FK.
37	usuario_registra_tipo_producto	tipo_producto.id_usuario_registra	1:N → FK.
38	usuario_registra_tipo_evento	tipo_evento.id_usuario_registra	1:N → FK.
39	tipo_servicio_tiene_servicio	servicio.id_tipo_servicio	1:N → FK.
40	usuario_ofrece_servicio	servicio.id_usuario	1:N → FK; repetía nombre, descripción y tarifa.
41	servicio_recibe_calificacion	calificacion → solicitud_servicio.id_servicio	1:N → FK a través de la solicitud.
42	usuario_califica	calificacion → solicitud_servicio.id_usuario_solicitante	Quien califica es quien contrató; repetía puntaje y reseña.
43	tipo_producto_tiene_producto	producto.id_tipo_producto	1:N → FK.
44	usuario_vende_producto	producto.id_usuario	1:N → FK; repetía nombre, precio y stock.
45	tipo_evento_tiene_evento	evento.id_tipo_evento	1:N → FK.
46	usuario_organiza_evento	evento.id_usuario_organizador	1:N → FK; repetía título, fecha y lugar.

Tabla EA3 nueva	Origen
solicitud_servicio	Relación N:M "solicita" del modelo conceptual (EA1) que la EA2 había omitido. Traza de servicios (EA3).
compra_producto	Relación N:M "compra" del modelo conceptual (EA1) que la EA2 había omitido. Traza de ventas (EA3).
participacion_evento	Relación N:M "participa" del modelo conceptual (EA1) que la EA2 había omitido. Traza de eventos (EA3).

3.- Tabla Comparativa de tipos de Datos

La tabla relaciona cada tipo de dato genérico del Diccionario de Datos Genérico (EA2) con el tipo de PostgreSQL 16 utilizado en la implementación, su dominio y su tamaño de almacenamiento.

Genérico	PostgreSQL	Bytes	Uso en el modelo
AUTOINCREMENTABLE	SMALLINT / INTEGER / BIGINT GENERATED ALWAYS AS IDENTITY	2 / 4 / 8	PK sustitutas de todas las entidades fuertes
UUID	UUID (gen_random_uuid())	16	No usado: las PK enteras (4 bytes) son suficientes

Genérico	PostgreSQL	Bytes	Uso en el modelo
ENTERO	SMALLINT / INTEGER / BIGINT	2 / 4 / 8	puntuacion, cantidad_disponible, cupo_maximo, intentos_fallidos
DECIMAL	NUMERIC(12,2)	variable	precio_servicio, precio_producto, valor_total, valor_entrada
LÓGICO	BOOLEAN	1	es_privado, fue_leido, asistio, es_nuevo
TEXTO	TEXT / VARCHAR(n)	variable	contenido_publicacion, cuerpo_mensaje (TEXT); descripciones (VARCHAR)
CARACTER	VARCHAR(n)	n + 1	estados cortos con 3 o más valores
FECHA	DATE	4	fecha_nacimiento, fecha_creacion
FECHA_HORA	TIMESTAMP	8	fecha_publicacion, fecha_compra, fecha_envio
IMAGEN	VARCHAR(255) (URL)	≤ 256	url_fotografia
AUDIO	VARCHAR(255) (URL) / BYTEA	variable	No usado en esta fase
URL	VARCHAR(255)	≤ 256	url_multimedia
ENUM	VARCHAR(n) + CHECK (IN ...)	variable	estado_usuario, tipo_reaccion, estado_compra
TEXTO_CORTO	VARCHAR(n)	n + 1	nombre_grupo, asunto_mensaje
BINARIO	BYTEA	variable	No usado: los archivos se guardan por URL
JSON	JSONB	variable	perfil.atributos_perfil

Tipos de Datos - Tabla Comparativa					
#	Tipo de Dato Genérico	Tipos de Dato - PosgreSQL	Dominio	Tamaño BYTES	Observaciones
1	AUTOINCREMENTABLE	SMALLINT / INTEGER / BIGINT GENERATED ALWAYS AS IDENTITY	Enteros 1 .. 32.767 / 2.147.483.647 / 9,2x10^18	2 / 4 / 8	Estándar SQL. Reemplaza SERIAL. Usado en todas las PK sustitutas (id_usuario INTEGER, id_publicacion BIGINT).
2	UUID	UUID	128 bits, ej. a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	16	Disponible con gen_random_uuid(). No se usa en el modelo: las PK enteras (4 bytes) son suficientes y ocupan menos espacio.
3	ENTERO	SMALLINT / INTEGER / BIGINT	-32.768..32.767 / ±2.147 millones / ±9,2x10^18	2 / 4 / 8	SMALLINT para catálogos y puntuación; INTEGER para cantidades y cupos; BIGINT para tablas de alto volumen.
4	DECIMAL	NUMERIC(p,s)	Hasta 131.072 dígitos antes del punto; exacto	Variable (≈ 2 por cada 4 dígitos + 3-8)	NUMERIC(12,2) para precios y valores en pesos: exacto, sin errores de redondeo de REAL/DOUBLE.
5	LÓGICO	BOOLEAN	TRUE / FALSE / NULL	1	Todo atributo de dos valores: es_privado, fue_leido, asistio, es_nuevo, estado_rol.
6	TEXTO	TEXT / VARCHAR(n)	Cadena de longitud variable (hasta 1 GB)	Variable (n + 1 a 4)	TEXT para contenido largo sin tope (publicaciones, mensajes); VARCHAR(n) cuando el negocio fija un máximo.
7	CARACTER	CHAR(n) / VARCHAR(n)	Cadena de longitud fija (CHAR) o limitada (VARCHAR)	n + 1	Se prefiere VARCHAR(n) para códigos y estados cortos: CHAR rellena con espacios.
8	FECHA	DATE	4713 a.C. .. 5874897 d.C., resolución 1 día	4	fecha_nacimiento, fecha_creacion del grupo.
9	FECHA_HORA	TIMESTAMP	4713 a.C. .. 294276 d.C., resolución 1 µs	8	Trazabilidad: fecha_publicacion, fecha_compra, fecha_envio. DEFAULT CURRENT_TIMESTAMP.
10	IMAGEN	VARCHAR(255) (URL) / BYTEA	Ruta o enlace a la imagen / binario	Variable	Se guarda la URL (url_fotografia, url_multimedia) y el archivo en almacenamiento externo; BYTEA engorda la BD.
11	AUDIO	VARCHAR(255) (URL) / BYTEA	Ruta o enlace al audio / binario	Variable	Mismo criterio que IMAGEN (no usado en el modelo actual).
12	URL	VARCHAR(255)	Cadena hasta 255 caracteres	Hasta 256	url_multimedia, url_fotografia.
13	ENUM	VARCHAR(n) + CHECK (campo IN (...))	Conjunto cerrado de valores	Variable	Se usa CHECK en lugar de CREATE TYPE ... AS ENUM porque agregar valores es más simple con ALTER TABLE.
14	TEXTO_CORTO (opcional)	VARCHAR(n)	Cadena de hasta n caracteres	n + 1	Nombres, títulos y asuntos: nombre_grupo VARCHAR(80), asunto_mensaje VARCHAR(150).
15	BINARIO (opcional)	BYTEA	Secuencia de bytes	Variable (hasta 1 GB)	No se usa: los archivos se referencian por URL.
16	JSON	JSONB	Documento JSON almacenado en binario	Variable	perfil.atributos_perfil (intereses, habilidades, deportes). JSONB permite índices GIN y consultas por clave.

Figura 4. Pestaña "Tipos de Dato": tabla comparativa de tipos genéricos y tipos de PostgreSQL.

4.- Diccionario de Datos Físico (SGBD PostgreSQL)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

El Diccionario de Datos Físico se elaboró en la pestaña "DICCIONARIO DATOS (Físico)" de la hoja de cálculo. Los nombres y descripciones de las tablas se escriben una sola vez en la pestaña del inventario corregido; el inventario del diccionario los toma por referencia de celda y cada tabla en detalle referencia a su vez al inventario, de modo que un cambio de nombre se propaga a todo el documento. Cada campo registra tipo de PostgreSQL, tamaño, nulidad, PK, FK, UK y una nota con el valor por defecto, la acción referencial y las restricciones CHECK. El diccionario se generó a partir del catálogo de la base de datos ya creada, por lo que coincide exactamente con el script DDL.

Justificación de los tipos de datos

- **VARCHAR(n) en lugar de TEXT:** se usa VARCHAR con tamaño cuando el negocio define un máximo (nombre_rol 30, correo_electronico 120, nombre_cuenta 30). El límite actúa como validación y documenta el dato. TEXT se reserva para contenido_publicacion y cuerpo_mensaje, que no deben cortarse. En PostgreSQL ambos se almacenan igual, por lo que la elección es de integridad y no de rendimiento.
- **numero_documento y telefono_usuario como VARCHAR:** son identificadores, no cantidades: no se opera con ellos, pueden tener ceros a la izquierda, letras (pasaporte) o el signo +.
- **SMALLINT, INTEGER o BIGINT:** SMALLINT (2 bytes, hasta 32.767) para catálogos que nunca tendrán miles de filas (rol, tipo_*) y para puntuacion; INTEGER (4 bytes, 2.147 millones) para usuario, grupo, servicio, producto y evento; BIGINT (8 bytes) para tablas de alto volumen que crecen con cada interacción: publicacion, comentario, mensaje, notificacion, solicitud_servicio y compra_producto. Las FK usan exactamente el mismo tipo que la PK referenciada.
- **NUMERIC(12,2) para dinero:** es exacto, a diferencia de REAL o DOUBLE PRECISION que acumulan errores de redondeo. 12 dígitos con 2 decimales permiten valores hasta 9.999.999.999,99 pesos.
- **TIMESTAMP con DEFAULT CURRENT_TIMESTAMP:** toda acción de la red guarda fecha y hora para ordenar y auditar. DATE se usa solo cuando la hora no aporta (fecha_nacimiento).
- **BOOLEAN para atributos de dos valores:** todo atributo con solo dos estados posibles es BOOLEAN (estado_rol, estado de los catálogos, es_privado, fue_leido, asistio, me_gusta, es_nuevo). Los estados con tres o más valores (solicitud, membresía, cuenta, producto, evento) usan VARCHAR con CHECK, porque un BOOLEAN solo representa dos.
- **JSONB:** en atributos_perfil, por ser semiestructurado (intereses, habilidades) y permitir consultas por clave e índices.

Justificación de las claves primarias

- **Entidades fuertes:** PK sustituta numérica GENERATED ALWAYS AS IDENTITY: es corta, estable y no cambia si el usuario corrige sus datos. Se prefiere IDENTITY sobre SERIAL porque es el estándar SQL y no permite insertar valores manuales por error.
- **Entidades débiles:** cuenta usa la PK del usuario (relación 1:1) y calificacion la PK de la solicitud; reaccion usa la PK compuesta (id_publicacion, id_usuario), que además impide reaccionar dos veces.
- **Relaciones N:M:** grupo_usuario, usuario_sigue_usuario y participacion_evento usan la combinación de las dos FK como PK compuesta, lo que impide duplicar la membresía, el seguimiento o la inscripción.

Descripción de las tablas implementadas

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

1. rol — Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_rol	SMALLINT	No	PK	Identificador único del rol
nombre_rol	VARCHAR(30)	No	UK	Nombre del rol: administrador, auxiliar, miembro, visitante
descripcion_rol	VARCHAR(255)	Sí		Alcance de los permisos que otorga el rol
estado_rol	BOOLEAN	No		Indica si el rol puede asignarse (TRUE = vigente)

PK: id_rol. Sustituta SMALLINT: catálogo pequeño.

2. tipo_usuario — Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_tipo_usuario	SMALLINT	No	PK	Identificador único del tipo de usuario
nombre_tipo_usuario	VARCHAR(40)	No	UK	Nombre del tipo: estudiante, docente, egresado…
descripcion_tipo_usuario	VARCHAR(255)	Sí		Aclaración del alcance del tipo de usuario
estado_tipo_usuario	BOOLEAN	No		Indica si el tipo está vigente para nuevas asignaciones

PK: id_tipo_usuario. Sustituta SMALLINT: catálogo pequeño.

3. usuario — Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_usuario	INTEGER	No	PK	Identificador interno del usuario
id_tipo_usuario	SMALLINT	No	FK	Tipo de usuario (vínculo con la Institución)
id_rol	SMALLINT	No	FK	Rol de permisos en la Red
numero_documento	VARCHAR(20)	No	UK	Documento de identidad
nombres_usuario	VARCHAR(60)	No		Nombres de la persona
apellidos_usuario	VARCHAR(60)	No		Apellidos de la persona
correo_electronico	VARCHAR(120)	No	UK	Correo de contacto, no repetible
telefono_usuario	VARCHAR(15)	Sí		Teléfono de contacto
fecha_nacimiento	DATE	Sí		Fecha de nacimiento (análisis demográfico)
fecha_registro	TIMESTAMP	No		Fecha y hora de registro en la Red
estado_usuario	VARCHAR(12)	No		Estado: activo, inactivo o suspendido

PK: id_usuario. Sustituta INTEGER; el documento y el correo quedan como UK porque pueden corregirse.

4. cuenta — Credenciales y estado de acceso del usuario, separadas de sus datos personales.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_usuario	INTEGER	No	PK/FK	Usuario dueño de la cuenta (PK y FK, relación 1:1)
nombre_cuenta	VARCHAR(30)	No	UK	Nombre de usuario para iniciar sesión
clave_cifrada	VARCHAR(255)	No		Hash de la contraseña (nunca texto plano)
estado_cuenta	VARCHAR(12)	No		Estado: activa, bloqueada o pendiente
fecha_ultimo_acceso	TIMESTAMP	Sí		Fecha y hora del último ingreso exitoso
intentos_fallidos	SMALLINT	No		Intentos fallidos consecutivos de acceso

PK: id_usuario. Misma PK del usuario: garantiza la relación 1:1 por diseño.

5. grupo — Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_grupo	INTEGER	No	PK	Identificador único del grupo
nombre_grupo	VARCHAR(80)	No	UK	Nombre del grupo, no repetible
descripcion_grupo	VARCHAR(500)	Sí		Propósito e intereses del grupo
fecha_creacion	DATE	No		Fecha de creación del grupo
es_privado	BOOLEAN	No		TRUE si el ingreso requiere aprobación
estado_grupo	VARCHAR(10)	No		Estado: activo, inactivo o cerrado

PK: id_grupo. Sustituta INTEGER; el nombre es UK.

6. perfil — Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_perfil	INTEGER	No	PK	Identificador único del perfil
id_usuario	INTEGER	Sí	FK/UK	Usuario dueño del perfil (vacío si es de grupo)
id_grupo	INTEGER	Sí	FK/UK	Grupo dueño del perfil (vacío si es de usuario)
titular_perfil	VARCHAR(100)	No		Título visible del perfil
area_estudio	VARCHAR(100)	Sí		Área de estudio o disciplina principal
descripcion_perfil	VARCHAR(500)	Sí		Presentación breve del titular
atributos_perfil	JSONB	Sí		Intereses, habilidades, deportes, libros (JSONB)
url_fotografia	VARCHAR(255)	Sí		Enlace a la foto de perfil
fecha_actualizacion	TIMESTAMP	No		Fecha y hora de la última modificación

PK: id_perfil. Sustituta INTEGER; id_usuario e id_grupo son UK para que cada titular tenga un solo perfil.

7. grupo_usuario — Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_grupo	INTEGER	No	PK/FK	Grupo al que pertenece (parte de la PK)
id_usuario	INTEGER	No	PK/FK	Usuario miembro (parte de la PK)
fecha_ingreso	TIMESTAMP	No		Fecha y hora de solicitud/ingreso
estado_membresia	VARCHAR(12)	No		Estado: pendiente, activa, rechazada, retirada
rol_en_grupo	VARCHAR(15)	No		Rol interno: administrador, moderador, miembro

PK: id_grupo, id_usuario. Compuesta: un usuario aparece una sola vez en cada grupo.

8. usuario_sigue_usuario — Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_usuario_seguidor	INTEGER	No	PK/FK	Usuario que sigue (parte de la PK)
id_usuario_seguido	INTEGER	No	PK/FK	Usuario seguido (parte de la PK)
fecha_seguimiento	TIMESTAMP	No		Fecha y hora en que empezó a seguir

PK: id_usuario_seguidor, id_usuario_seguido. Compuesta: no se puede seguir dos veces a la misma persona.

9. publicacion — Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_publicacion	BIGINT	No	PK	Identificador único de la publicación

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_usuario	INTEGER	No	FK	Usuario autor
id_grupo	INTEGER	Sí	FK	Grupo donde se publica (vacío = muro general)
titulo_publicacion	VARCHAR(150)	Sí		Encabezado corto
contenido_publicacion	TEXT	No		Texto completo de la publicación
url_multimedia	VARCHAR(255)	Sí		Enlace al archivo adjunto (imagen o video)
fecha_publicacion	TIMESTAMP	No		Fecha y hora de publicación
estado_publicacion	VARCHAR(10)	No		Estado: visible, oculto, eliminado

PK: id_publicacion. Sustituta BIGINT: tabla de alto volumen.

10. comentario — Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_comentario	BIGINT	No	PK	Identificador único del comentario
id_publicacion	BIGINT	No	FK	Publicación comentada
id_usuario	INTEGER	No	FK	Usuario que comenta
texto_comentario	VARCHAR(1000)	No		Texto del comentario
fecha_comentario	TIMESTAMP	No		Fecha y hora del comentario
estado_comentario	VARCHAR(10)	No		Estado: visible, oculto, eliminado

PK: id_comentario. Sustituta BIGINT generada por el SGBD (evita calcular consecutivos).

11. reaccion — Reacción breve (me gusta, me encanta...) de un usuario a una publicación; una sola por usuario y publicación.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_publicacion	BIGINT	No	PK/FK	Publicación reaccionada (parte de la PK)
id_usuario	INTEGER	No	PK/FK	Usuario que reacciona (parte de la PK)
tipo_reaccion	VARCHAR(12)	No		Tipo: me_gusta, me_encanta, me_divierte, me_sorprende
fecha_reaccion	TIMESTAMP	No		Fecha y hora de la reacción

PK: id_publicacion, id_usuario. Compuesta: una reacción por usuario y publicación.

12. mensaje — Mensaje privado enviado por un usuario a otro dentro de la plataforma.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_mensaje	BIGINT	No	PK	Identificador único del mensaje
id_usuario_remitente	INTEGER	No	FK	Usuario que envía
id_usuario_destinatario	INTEGER	No	FK	Usuario que recibe
asunto_mensaje	VARCHAR(150)	Sí		Asunto breve
cuerpo_mensaje	TEXT	No		Contenido del mensaje
fecha_envio	TIMESTAMP	No		Fecha y hora de envío
fue_leido	BOOLEAN	No		TRUE si el destinatario ya lo leyó

PK: id_mensaje. Sustituta BIGINT: tabla de alto volumen.

13. notificacion — Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_notificacion	BIGINT	No	PK	Identificador único de la notificación

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_usuario	INTEGER	No	FK	Usuario destinatario
tipo_notificacion	VARCHAR(15)	No		Origen: comentario, reaccion, mensaje, seguidor, grupo, evento, servicio, producto
referencia_origen	BIGINT	Sí		Id del registro que originó el aviso
texto_notificacion	VARCHAR(255)	No		Texto que se muestra al usuario
fecha_generacion	TIMESTAMP	No		Fecha y hora de generación
fue_leida	BOOLEAN	No		TRUE si ya fue consultada

PK: id_notificacion. Sustituta BIGINT: tabla de alto volumen.

14. tipo_servicio — Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC...).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_tipo_servicio	SMALLINT	No	PK	Identificador del tipo de servicio
id_usuario_registra	INTEGER	No	FK	Administrador/auxiliar que lo registró
nombre_tipo_servicio	VARCHAR(60)	No	UK	Nombre de la categoría
descripcion_tipo_servicio	VARCHAR(255)	Sí		Aclaración del alcance
estado_tipo_servicio	BOOLEAN	No		TRUE si está vigente

PK: id_tipo_servicio. Sustituta SMALLINT: catálogo.

15. servicio — Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_servicio	INTEGER	No	PK	Identificador del servicio
id_usuario	INTEGER	No	FK	Usuario que ofrece el servicio
id_tipo_servicio	SMALLINT	No	FK	Categoría del servicio
nombre_servicio	VARCHAR(100)	No		Nombre comercial del servicio
descripcion_servicio	VARCHAR(1000)	Sí		Detalle de lo que incluye
precio_servicio	NUMERIC(12,2)	No		Valor cobrado (pesos colombianos)
duracion_minutos	INTEGER	Sí		Duración estimada en minutos
ubicacion_servicio	VARCHAR(150)	Sí		Lugar donde se presta
esta_disponible	BOOLEAN	No		TRUE si está activo para ser solicitado
fecha_publicacion	TIMESTAMP	No		Fecha y hora de publicación de la oferta

PK: id_servicio. Sustituta INTEGER.

16. solicitud_servicio — Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_solicitud	BIGINT	No	PK	Identificador de la solicitud (traza)
id_servicio	INTEGER	No	FK	Servicio solicitado
id_usuario_solicitante	INTEGER	No	FK	Usuario que contrata
fecha_solicitud	TIMESTAMP	No		Fecha y hora de la solicitud
estado_solicitud	VARCHAR(12)	No		Estado: pendiente, aceptada, rechazada, finalizada, cancelada
valor_acordado	NUMERIC(12,2)	No		Valor acordado entre las partes

PK: id_solicitud. Sustituta BIGINT: el mismo usuario puede contratar el mismo servicio varias veces.

17. calificacion — Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_solicitud	BIGINT	No	PK/FK	Solicitud calificada (PK y FK, 1 calificación por contratación)
puntuacion	SMALLINT	No		Puntuación de 1 a 5
texto_calificacion	VARCHAR(500)	Sí		Reseña escrita
fecha_calificacion	TIMESTAMP	No		Fecha y hora de la calificación

PK: id_solicitud. Misma PK de la solicitud: una calificación por contratación.

18. tipo_producto — Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets...).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_tipo_producto	SMALLINT	No	PK	Identificador del tipo de producto
id_usuario_registra	INTEGER	No	FK	Administrador que lo registró
nombre_tipo_producto	VARCHAR(60)	No	UK	Nombre de la categoría
descripcion_tipo_producto	VARCHAR(255)	Sí		Aclaración del alcance
estado_tipo_producto	BOOLEAN	No		TRUE si está vigente

PK: id_tipo_producto. Sustituta SMALLINT: catálogo.

19. producto — Artículo que un usuario publica para la venta, con precio, existencias y condición.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_producto	INTEGER	No	PK	Identificador del producto
id_usuario	INTEGER	No	FK	Usuario vendedor
id_tipo_producto	SMALLINT	No	FK	Categoría del producto
nombre_producto	VARCHAR(100)	No		Nombre del artículo
descripcion_producto	VARCHAR(1000)	Sí		Características del artículo
precio_producto	NUMERIC(12,2)	No		Precio unitario de venta
cantidad_disponible	INTEGER	No		Unidades disponibles
es_nuevo	BOOLEAN	No		TRUE si el producto es nuevo; FALSE si es usado
fecha_publicacion	TIMESTAMP	No		Fecha y hora de publicación
estado_producto	VARCHAR(12)	No		Estado: disponible, reservado, vendido, retirado

PK: id_producto. Sustituta INTEGER.

20. compra_producto — Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_compra	BIGINT	No	PK	Identificador de la compra (traza de venta)
id_producto	INTEGER	No	FK	Producto comprado
id_usuario_comprador	INTEGER	No	FK	Usuario comprador
fecha_compra	TIMESTAMP	No		Fecha y hora de la compra
cantidad	INTEGER	No		Unidades compradas
precio_unitario	NUMERIC(12,2)	No		Precio unitario al momento de la compra
valor_total	NUMERIC(14,2)	No		Total = cantidad x precio_unitario
estado_compra	VARCHAR(12)	No		Estado: pagada, entregada, cancelada

PK: id_compra. Sustituta BIGINT: cada compra es un hecho distinto aunque se repita comprador y producto.

21. tipo_evento — Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto…).

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_tipo_evento	SMALLINT	No	PK	Identificador del tipo de evento
id_usuario_registra	INTEGER	No	FK	Administrador que lo registró
nombre_tipo_evento	VARCHAR(60)	No	UK	Nombre de la categoría
descripcion_tipo_evento	VARCHAR(255)	Sí		Aclaración del alcance
estado_tipo_evento	BOOLEAN	No		TRUE si está vigente

PK: id_tipo_evento. Sustituta SMALLINT: catálogo.

22. evento — Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_evento	INTEGER	No	PK	Identificador del evento
id_usuario_organizador	INTEGER	No	FK	Usuario que organiza
id_tipo_evento	SMALLINT	No	FK	Categoría del evento
nombre_evento	VARCHAR(120)	No		Nombre del evento
descripcion_evento	VARCHAR(1000)	Sí		Detalle y agenda
fecha_inicio	TIMESTAMP	No		Fecha y hora de inicio
fecha_fin	TIMESTAMP	No		Fecha y hora de finalización
lugar_evento	VARCHAR(150)	No		Lugar físico o enlace virtual
cupo_maximo	INTEGER	Sí		Número máximo de participantes
valor_entrada	NUMERIC(12,2)	No		Valor de la entrada (0 = gratuito)
estado_evento	VARCHAR(12)	No		Estado: programado, en_curso, finalizado, cancelado

PK: id_evento. Sustituta INTEGER.

23. participacion_evento — Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.

Campo	Tipo (tamaño)	Nulo	Clave	Descripción
id_evento	INTEGER	No	PK/FK	Evento (parte de la PK)
id_usuario	INTEGER	No	PK/FK	Usuario participante (parte de la PK)
fecha_inscripcion	TIMESTAMP	No		Fecha y hora de inscripción
valor_pagado	NUMERIC(12,2)	No		Valor pagado por la entrada
asistio	BOOLEAN	No		TRUE si asistió
me_gusta	BOOLEAN	No		TRUE si dio 'me gusta' después del evento
comentario_evento	VARCHAR(500)	Sí		Comentario posterior al evento
fecha_hora_comentario	TIMESTAMP	Sí		Fecha y hora del comentario

PK: id_evento, id_usuario. Compuesta: un usuario se inscribe una vez a cada evento.

Implementación de las relaciones (claves foráneas)

Las 34 claves foráneas se declaran dentro de cada CREATE TABLE con nombre propio (fk_<tabla>_<destino>). Todas usan ON UPDATE CASCADE (si una PK cambiara, el cambio se propaga). La acción ON DELETE se eligió según el tipo de dependencia: RESTRICT para catálogos y para todo lo que es autoría o traza (ventas, mensajes, publicaciones), porque borrar el padre destruiría el histórico; en esos casos el registro se desactiva con su campo

de estado (borrado lógico). CASCADE para los datos que no tienen sentido sin su padre (cuenta, perfil, membresías, reacciones, notificaciones). SET NULL solo en publicacion.id_grupo: si un grupo desaparece, la publicación pasa al muro general.

Tabla padre	Tabla hija	Campo FK	ON DELETE	ON UPDATE	Justificación
rol	usuario	id_rol	RESTRICT	CASCADE	Catálogo: no se borra un rol asignado.
tipo_usuario	usuario	id_tipo_usuario	RESTRICT	CASCADE	Catálogo: no se borra una categoría en uso.
usuario	cuenta	id_usuario	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
grupo	perfil	id_grupo	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	perfil	id_usuario	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
grupo	grupo_usuario	id_grupo	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	grupo_usuario	id_usuario	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	usuario_sigue_usuario	id_usuario_seguido	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	usuario_sigue_usuario	id_usuario_seguidor	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
grupo	publicacion	id_grupo	SET NULL	CASCADE	Opcional: si se borra el grupo, la publicación queda en el muro general.
usuario	publicacion	id_usuario	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
publicacion	comentario	id_publicacion	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	comentario	id_usuario	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
publicacion	reaccion	id_publicacion	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	reaccion	id_usuario	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	mensaje	id_usuario_destinatario	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
usuario	mensaje	id_usuario_remitente	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
usuario	notificacion	id_usuario	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	tipo_servicio	id_usuario_registra	RESTRICT	CASCADE	Auditoría: se conserva quién creó la categoría.
tipo_servicio	servicio	id_tipo_servicio	RESTRICT	CASCADE	Catálogo: no se borra una categoría en uso.
usuario	servicio	id_usuario	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
servicio	solicitud_servicio	id_servicio	RESTRICT	CASCADE	Traza: no se borra un servicio contratado (esta_disponible = FALSE).
usuario	solicitud_servicio	id_usuario_solicitante	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
solicitud_servicio	calificacion	id_solicitud	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	tipo_producto	id_usuario_registra	RESTRICT	CASCADE	Auditoría: se conserva quién creó la categoría.
tipo_producto	producto	id_tipo_producto	RESTRICT	CASCADE	Catálogo: no se borra una categoría en uso.
usuario	producto	id_usuario	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
producto	compra_producto	id_producto	RESTRICT	CASCADE	Traza de venta: no se borra un producto vendido (se marca retirado).
usuario	compra_producto	id_usuario_comprador	RESTRICT	CASCADE	Traza de venta: no se pierde el comprador.
usuario	tipo_evento	id_usuario_registra	RESTRICT	CASCADE	Auditoría: se conserva quién creó la

Tabla padre	Tabla hija	Campo FK	ON DELETE	ON UPDATE	Justificación
					categoría.
tipo_evento	evento	id_tipo_evento	RESTRICT	CASCADE	Catálogo: no se borra una categoría en uso.
usuario	evento	id_usuario_organizador	RESTRICT	CASCADE	Autoría/traza: se conserva el histórico; el padre se desactiva por estado.
evento	participacion_evento	id_evento	CASCADE	CASCADE	Dependiente: no existe sin el padre, se elimina con él.
usuario	participacion_evento	id_usuario	RESTRICT	CASCADE	Traza de pago y asistencia: no se pierde al borrar un usuario.

Tablas pivote (N:M): grupo_usuario (padres: grupo y usuario), usuario_sigue_usuario (padre: usuario dos veces, como seguidor y seguido) y participacion_evento (padres: evento y usuario). En las tres, las dos FK forman la PK compuesta.

DICCIONARIO DE DATOS FÍSICO (PostgreSQL)

Inventario de Tablas

#	Tabla	Descripción Tabla	Observaciones
1	rol	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador	Entidad
2	tipo_usuario	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado	Entidad
3	usuario	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataf	Entidad
4	cuenta	Credenciales y estado de acceso del usuario, separadas de sus datos personales.	Entidad débil (1:1)
5	grupo	Comunidad de usuarios reunida por un interés común: grupos de estudio, semillerí	Entidad
6	perfil	Información pública e intereses de un usuario o de un grupo; incluye atributos se	Entidad
7	grupo_usuario	Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del g	Relación N:M
8	usuario_sigue_usuario	Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).	Relación N:M recursiva
9	publicacion	Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo	Entidad
10	comentario	Respuesta que un usuario escribe sobre una publicación; el autor puede ser distint	Entidad
11	reaccion	Reacción breve (me gusta, me encanta…) de un usuario a una publicación; una sol	Entidad débil
12	mensaje	Mensaje privado enviado por un usuario a otro dentro de la plataforma.	Entidad
13	notificacion	Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le conc	Entidad
14	tipo_servicio	Catálogo de categorías de servicio definido por la administración (asesoría, tutoría	Entidad
15	servicio	Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubi	Entidad
16	solicitud_servicio	Traza de cada contratación de un servicio por parte de otro usuario, con estado y v	Relación N:M con atributos (entidad asociativa)
17	calificacion	Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.	Entidad débil
18	tipo_producto	Catálogo de categorías de producto definido por la administración (papelería, libro	Entidad
19	producto	Artículo que un usuario publica para la venta, con precio, existencias y condición.	Entidad
20	compra_producto	Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.	Relación N:M con atributos (entidad asociativa)
21	tipo_evento	Catálogo de categorías de evento definido por la administración (congreso, taller, f	Entidad
22	evento	Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y v	Entidad
23	participacion_evento	Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario	Relación N:M con atributos

Tablas en Detalle

1	Tabla	rol	Descripción Tabla	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).				Versión	3.0
#	Campo	Descripción Campo	Tipo Dato (PostgreSQL)	Tamaño / Longitud	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	SMALLINT	2 bytes	No	S			GENERATED ALWAYS AS IDENTITY
2	nombre_rol	Nombre del rol: administrador, auxiliar, miembro, visitante	VARCHAR	30	No			S	
3	descripcion_rol	Alcance de los permisos que otorga el rol	VARCHAR	255	Sí				
4	estado_rol	Indica si el rol puede asignarse (TRUE = vigente)	BOOLEAN	1 byte	No				DEFAULT true

Figura 5. Parte inicial del Diccionario de Datos Físico: inventario de tablas por referencia y detalle de la primera tabla.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos

El script 20262-bd1-ea3-equipo-A-scripts-crea.sql contiene las 23 sentencias CREATE TABLE, numeradas igual que el inventario. Se ejecuta completo en pgAdmin4 (Query Tool) sobre una base de datos vacía, sin errores, y fue probado en PostgreSQL 16. Las tablas se crean en siete bloques: primero las independientes y después las que dependen de ellas, de modo que toda tabla referenciada por una FK ya existe al momento de crearla.

Cada CREATE TABLE sigue la misma estructura: (1) columnas con tipo, tamaño, NOT NULL y DEFAULT; (2) CONSTRAINT pk_ con la clave primaria; (3) CONSTRAINT uk_ para valores únicos; (4) CONSTRAINT fk_ con REFERENCES, ON DELETE y ON UPDATE; (5) CONSTRAINT ck_ con las reglas de negocio. Nombrar las restricciones permite identificarlas en los mensajes de error y modificarlas después con ALTER TABLE.

Bloque	Tablas (en orden de ejecución)	Por qué en ese orden
1	rol, tipo_usuario	Catálogos sin FK.
2	usuario, cuenta	usuario depende de los catálogos; cuenta de usuario.
3	grupo, perfil, grupo_usuario, usuario_sigue_usuario	Dependen de usuario y grupo.
4	publicacion, comentario, reaccion, mensaje, notificacion	Contenido creado por usuarios.
5	tipo_servicio, servicio, solicitud_servicio, calificacion	Cadena catálogo → oferta → contratación → calificación.
6	tipo_producto, producto, compra_producto	Cadena catálogo → producto → venta.
7	tipo_evento, evento, participacion_evento	Cadena catálogo → evento → inscripción.

Ejemplo de la estructura de una instrucción CREATE TABLE del script:

```
CREATE TABLE compra_producto (
    id_compra             BIGINT        GENERATED ALWAYS AS IDENTITY,
    id_producto           INTEGER       NOT NULL,
    id_usuario_comprador  INTEGER       NOT NULL,
    fecha_compra          TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    cantidad              INTEGER       NOT NULL,
    precio_unitario       NUMERIC(12,2) NOT NULL,
    valor_total           NUMERIC(14,2) NOT NULL,
    estado_compra         VARCHAR(12)   NOT NULL DEFAULT 'pagada',
    CONSTRAINT pk_compra_producto PRIMARY KEY (id_compra),
    CONSTRAINT fk_compra_producto_producto FOREIGN KEY (id_producto)
        REFERENCES producto (id_producto) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_compra_producto_usuario FOREIGN KEY (id_usuario_comprador)
        REFERENCES usuario (id_usuario) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT ck_compra_producto_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_compra_producto_total CHECK (valor_total = cantidad * precio_unitario),
    ...
);
```

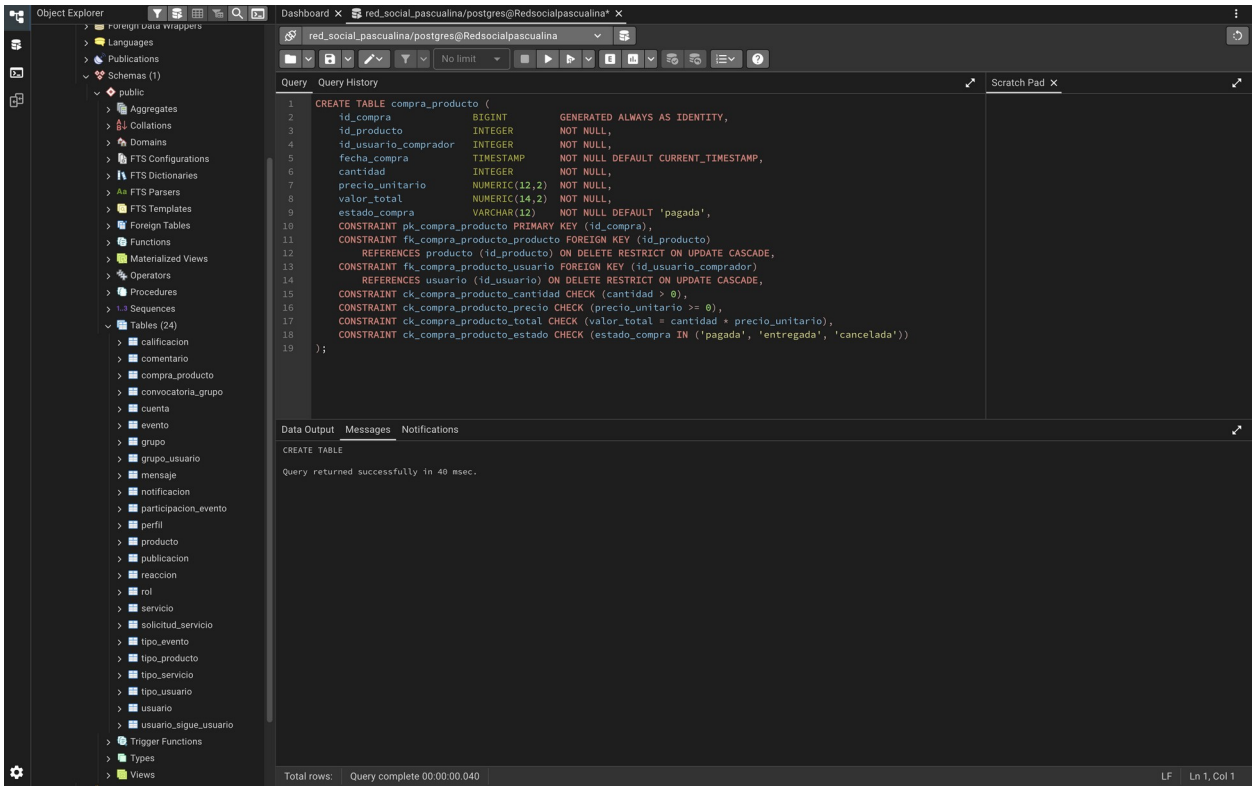


Figura 6. Ejecución en pgAdmin4 del CREATE TABLE de compra_producto: "CREATE TABLE - Query returned successfully".

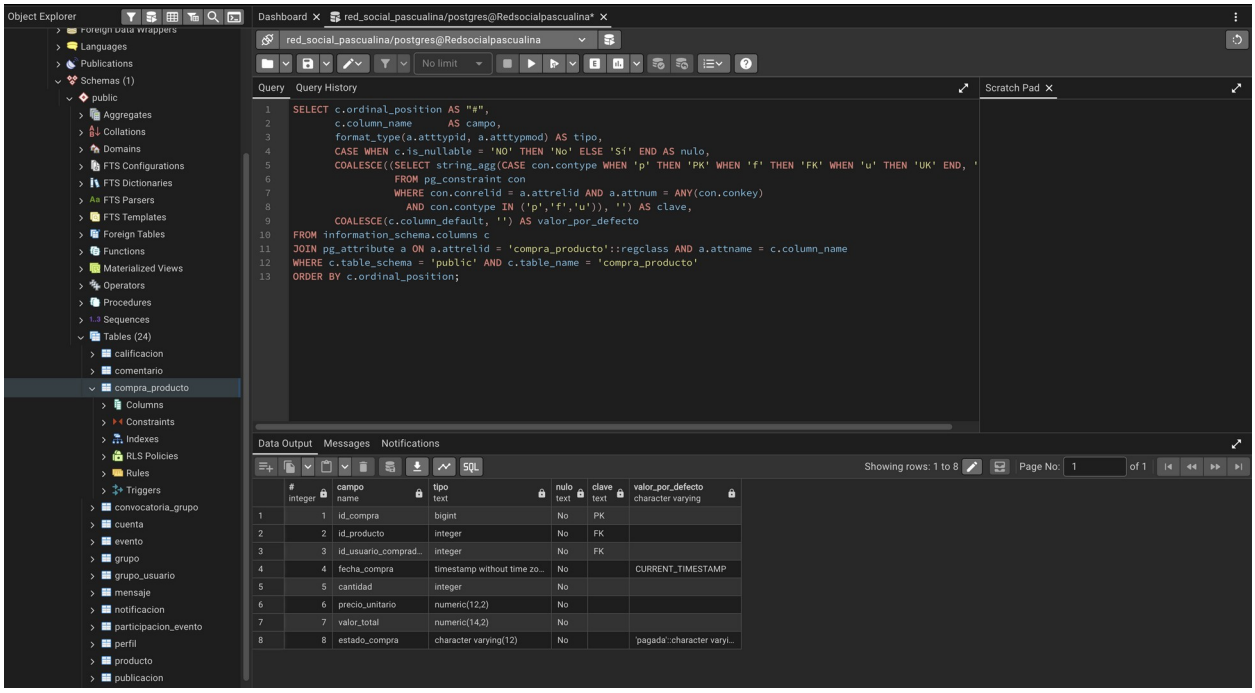


Figura 7. Resultado: estructura de compra_producto consultada desde el catálogo (campos, tipos, nulidad, PK/FK y valores por defecto). El árbol muestra 24 tablas: las 23 del script de creación más convocatoria_grupo, creada por el script de modificación.

6.- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

El script 20262-bd1-ea3-equipo-A-scripts-modifica.sql se ejecuta después del de creación. Modifica la tabla usuario y gestiona una tabla nueva propuesta por el equipo, convocatoria, que registra las convocatorias que publican los grupos (semilleros, hackatones, monitorías) y luego se renombra a convocatoria_grupo. Todas las instrucciones se ejecutan en secuencia sin errores.

Punto	Instrucción	Qué hace
1.1	ALTER TABLE usuario ADD COLUMN programa_academico VARCHAR(80)	Agrega el programa académico del usuario.
1.2	ALTER TABLE usuario ALTER COLUMN telefono_usuario TYPE VARCHAR(20)	Amplía el teléfono para indicativo internacional.
1.3.1	CREATE TABLE convocatoria ()	Crea la tabla nueva (PostgreSQL permite crearla sin columnas).
1.3.2	ADD COLUMN id_convocatoria … IDENTITY + PRIMARY KEY; titulo (VARCHAR), descripcion (TEXT), numero_plazas (INTEGER)	Agrega la PK y tres campos (texto y numérico).
1.3.3	DROP COLUMN descripcion_convocatoria	Quita un campo.
1.3.4	RENAME TO convocatoria_grupo; RENAME CONSTRAINT pk_…	Renombra la tabla y su PK.
1.3.5	ADD COLUMN codigo_convocatoria + CONSTRAINT UNIQUE	Agrega un campo único.
1.3.6	ADD fecha_inicio, fecha_fin + CHECK (fecha_fin >= fecha_inicio)	Fechas con control de orden.
1.3.7	ADD numero_postulados INTEGER + CHECK (>= 0)	Entero no negativo.
1.3.8	ALTER COLUMN titulo_convocatoria TYPE VARCHAR(150)	Cambia el tamaño de un campo texto.
1.3.7 (bis)	numero_plazas TYPE SMALLINT, NOT NULL + CHECK BETWEEN 1 AND 100	Modifica el numérico con control de rango.
1.3.8 (bis)	CREATE INDEX ix_convocatoria_grupo_fecha_inicio	Índice para consultas por fecha.
1.3.9	DROP CONSTRAINT ck_…_fechas; DROP COLUMN fecha_fin	Elimina una fecha (primero su restricción).
1.3.10	TRUNCATE TABLE convocatoria_grupo RESTART IDENTITY	Borra todos los datos sin traza fila a fila y reinicia el contador (antes se insertan 2 filas de prueba para evidenciarlo).

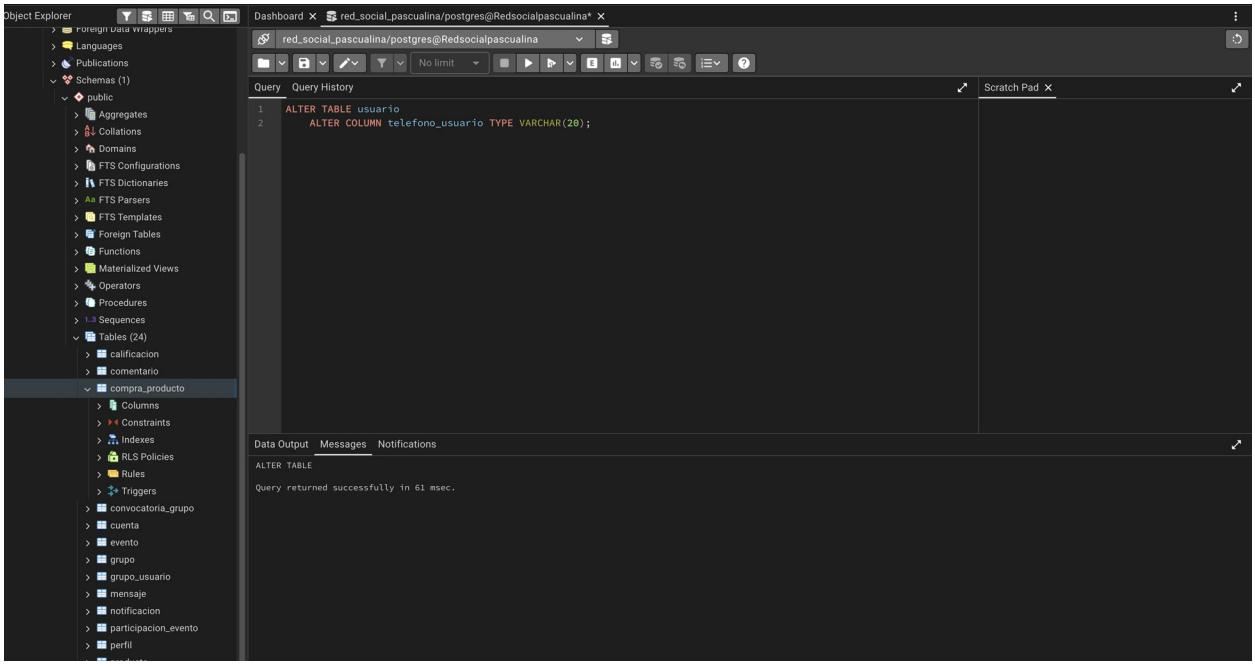


Figura 8. Ejecución en pgAdmin4 de la modificación 1.2 (ALTER TABLE usuario ALTER COLUMN telefono_usuario TYPE VARCHAR(20)).

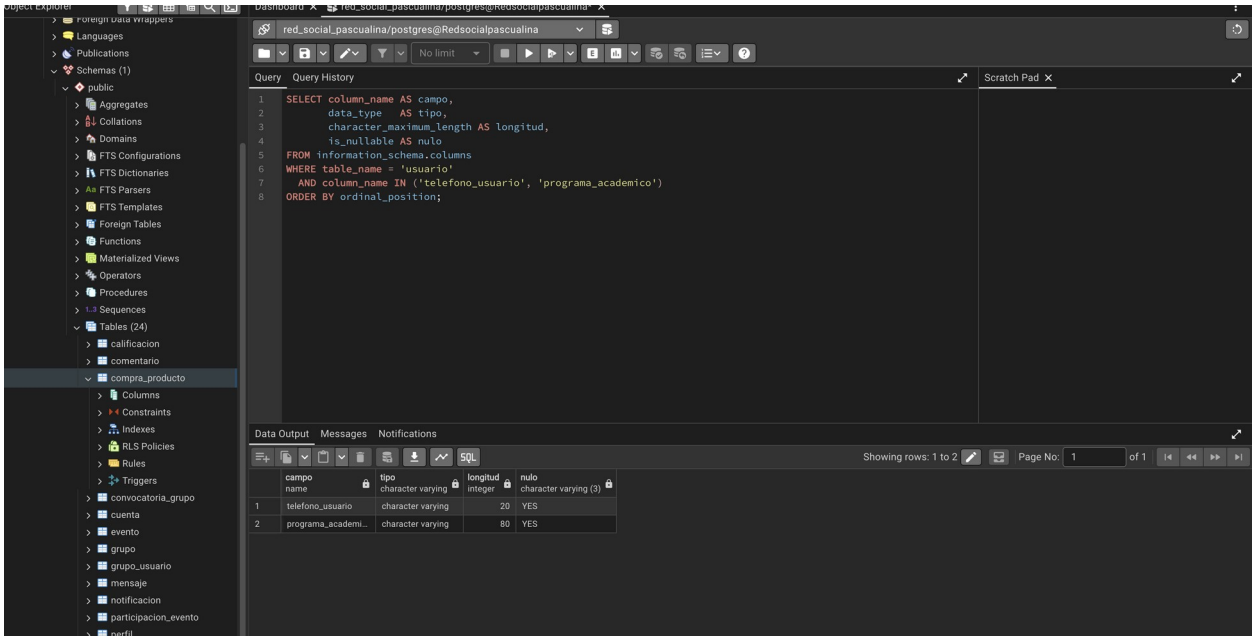


Figura 9. Verificación en usuario: telefono_usuario quedó en VARCHAR(20) y programa_academico (punto 1.1) en VARCHAR(80).

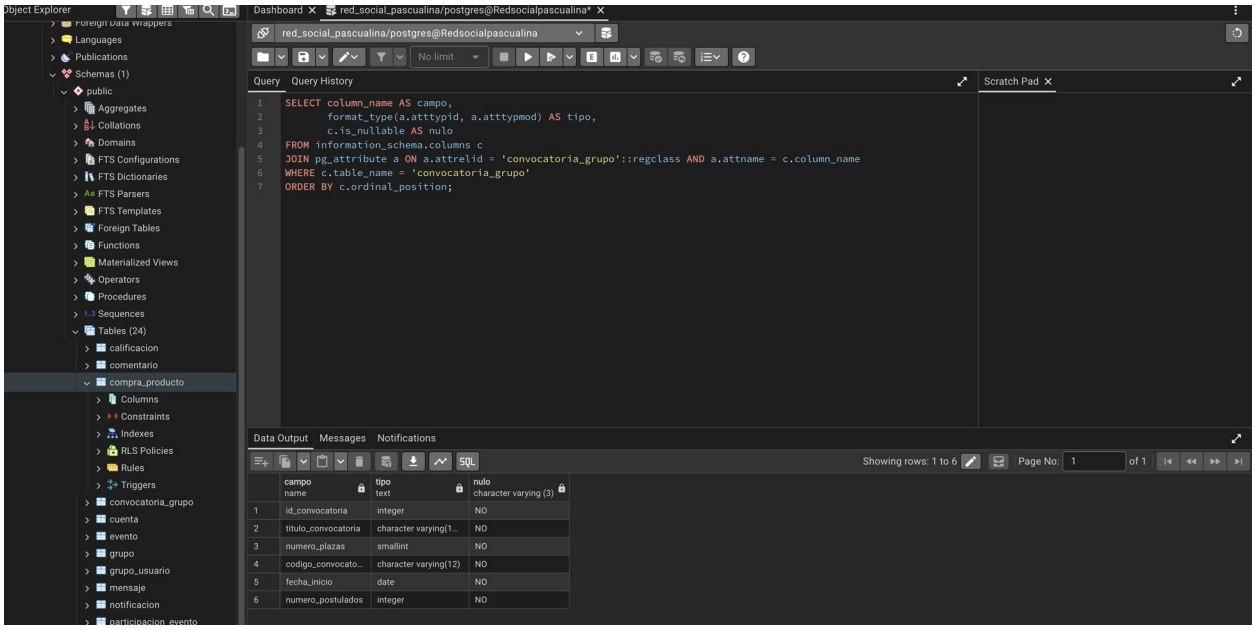


Figura 10. Estructura final de convocatoria_grupo después de los puntos 1.3.1 a 1.3.10.

7.- Análisis de resultados

La Fase 3 permitió comprobar en un SGBD real si el diseño de las fases anteriores era correcto, y la respuesta inicial fue que no del todo. Al intentar llevar a DDL el modelo lógico de la EA2 aparecieron problemas que en papel pasaban desapercibidos: una clave foránea que apuntaba a una columna que no existía (id_cuenta), una referencia circular entre usuario y tipo_usuario que impedía crear las tablas en orden, y 23 tablas que repetían los mismos atributos porque relaciones 1:N se habían convertido en tablas. El modelo físico obligó a volver al modelo conceptual de la EA1 y a corregirlo, de modo que hoy los tres niveles son consistentes: cada entidad del

diagrama de Chen es una tabla, cada relación 1:N es una clave foránea y cada relación N:M con atributos es una tabla pivote.

Respecto a los tipos de datos, la traducción del diccionario genérico a PostgreSQL exigió decisiones concretas. El tipo genérico ENTERO se separó en SMALLINT, INTEGER y BIGINT según el volumen esperado de cada tabla; DECIMAL se implementó como NUMERIC(12,2) para que los valores de compras y servicios sean exactos; TEXTO se dividió entre VARCHAR(n), cuando existe un límite de negocio, y TEXT, cuando el contenido es libre. Los tipos ENUM se implementaron con restricciones CHECK y el atributo semiestructurado del perfil como JSONB. Se cuidó que cada FK tuviera exactamente el mismo tipo que la PK que referencia.

En las relaciones, el aporte principal de esta fase fueron las acciones referenciales. Se distinguieron tres casos: datos dependientes que se eliminan con su padre (CASCADE), datos de autoría y traza que deben conservarse (RESTRICT con borrado lógico por estado) y una relación opcional (SET NULL). Esto responde a la exigencia del enunciado de dejar traza de las ventas de productos, servicios y eventos: compra_producto, solicitud_servicio y participacion_evento no pueden perderse al eliminar un usuario o un producto.

Las restricciones CHECK trasladaron reglas del caso de estudio a la base de datos: un usuario no puede seguirse a sí mismo ni enviarse mensajes, la fecha de fin de un evento no puede ser anterior a la de inicio, la puntuación está entre 1 y 5 y el valor total de una compra debe ser igual a la cantidad por el precio unitario. Se probaron insertando datos válidos e inválidos, y el SGBD rechazó los inválidos. El resultado final es un modelo de 23 tablas, 34 claves foráneas y un script que se ejecuta de principio a fin sin errores, con un diccionario físico que coincide con la base creada porque se obtuvo de su propio catálogo.

8.- Reflexiones y conclusiones individuales

Juan Camilo González Hoyos

Esta fase fue la que más me enseñó del curso, porque fue la primera vez que el modelo tuvo que funcionar de verdad. En las fases anteriores un error en una tabla podía pasar desapercibido en el documento; en pgAdmin4 no, porque el script se detiene en la primera clave foránea que apunta a algo que no existe. Me di cuenta de que nuestro modelo lógico de la Fase 2 tenía errores serios, como tablas que repetían los mismos campos de otras y una referencia a una columna inexistente, y que la causa no era técnica sino de proceso: trabajamos por separado, con versiones distintas del diagrama, y nadie comparó el resultado final contra el modelo conceptual.

Lo más valioso que aprendí fue la diferencia entre una relación 1:N y una N:M en el momento de implementarlas. Una relación 1:N no es una tabla, es una columna con una clave foránea; solo las relaciones N:M, o las que generan transacciones como una compra, necesitan una tabla propia. También entendí que las acciones ON DELETE no son un detalle de sintaxis sino una decisión de negocio: decidir que una compra no se puede borrar cuando se elimina un producto es decidir que la red social conserva su historial de ventas. Lo mismo con los tipos de datos: elegir NUMERIC en lugar de un tipo de punto flotante para el dinero, o VARCHAR con un límite para el correo, es poner reglas del negocio dentro de la base de datos.

En lo profesional, esto se relaciona directamente con lo que hago fuera de la universidad desarrollando software: una base de datos mal diseñada se termina pagando en el backend con validaciones duplicadas, datos inconsistentes y consultas lentas. Ahora reviso primero el modelo de datos antes de escribir una línea de código de la aplicación.

Sobre el trabajo en equipo, la lección más clara es que un proyecto de datos necesita una sola fuente de verdad. Para lo que queda del semestre nos propusimos trabajar sobre un único archivo versionado en el repositorio, revisar cada entrega contra el modelo anterior antes de enviarla y repartir el trabajo por bloques de tablas, no

por documentos sueltos. El caso de la Red Social Pascualina me mostró que el modelo de datos es la base de todo el sistema: si el modelo es coherente, las consultas, los reportes y la aplicación se construyen encima sin contradicciones; si no lo es, cada fase hereda los errores de la anterior.

9.- Informe (calidad de presentación y respeto de los requerimientos)

Entregables de la tarea (carpeta Tarea3 del repositorio):

- tarea3/informe/20262-bd1-ea3-equipo-A-informe.pdf
- tarea3/resultados/20262-bd1-ea3-equipo-A-resultados.xlsx (diagrama, inventario, tipos de dato y diccionario físico)
- tarea3/resultados/20262-bd1-ea3-equipo-A-scripts-crea.sql
- tarea3/resultados/20262-bd1-ea3-equipo-A-scripts-modifica.sql
- tarea3/video: enlace al video 20262-bd1-ea3-equipo-A-video

10.- Video de Sustentación y defensa “en vivo”

El video de sustentación con la ejecución en vivo en pgAdmin4 se presentará en la Tarea 4, según lo indicado por el docente.

11.- Repositorio GIT

Enlace al repositorio:

https://github.com/ZeikoDev/bd1-20262-g051-equipo-A/tree/main/Tarea3

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea		Peso	Cal
1	Diagrama Entidad-Relación de Chen (corregido)		20	
2	Inventario de tablas resultantes del Modelo Lógico (corregido)		10	
3	Tabla Comparativa de tipos de Datos		10	
4	Diccionario de Datos Físico (PostgreSQL)		100	
5	DDL - Scripts de CREACIÓN de Bases de Datos		120	
6	DDL - Scripts de MODIFICACIÓN de Bases de Datos		30	
7	Análisis de resultados (en equipo - 300 palabras mínimo)		20	
8	Reflexiones y Conclusiones individuales (300 palabras mínimo)		30	
9	Informe (doc) y resultados (hoja de cálculo): Estructura, Contenido y Calidad de presentación. NOTA: Borrar instrucciones		40	
10	Video de Sustentación y defensa “en vivo”		200	
11	Repositorio GIT		20	
	NOTA = xx/600 =	Total	600	